
3.1 Manipulación Avanzada del Objeto `String`

En Dart, los objetos `String` son secuencias inmutables de unidades de código UTF-16. Cada vez que modificamos una cadena mediante concatenación o sustitución, en realidad creamos una nueva instancia en memoria.

Métodos Clave para Inserción, Búsqueda y Limpieza

- `.trim()`, `.trimLeft()`, `.trimRight()`: Eliminan espacios en blanco al inicio o al final, esencial para sanitizar entradas de texto.
- `.contains(Pattern)` / `.startsWith()` / `.endsWith()`: Verificaciones booleanas rápidas sobre el contenido de la cadena.
- `.replaceAll(Pattern, String)`: Sustitución global utilizando texto plano o Expresiones Regulares (RegExp).
- `.substring(int start, [int? end])`: Extrae una subcadena a partir del índice inicial hasta el final opcional.

Construcción Eficiente con `StringBuffer`

Cuando necesitamos concatenar cadenas de manera reiterada (por ejemplo, en bucles masivos o en la generación de reportes), realizar `cadena += "nuevo texto"` genera múltiples instancias intermedias en memoria, sobrecargando el recolector de basura.

Para evitar esto, utilizamos la clase `StringBuffer`, la cual almacena el contenido en un búfer interno contiguo y construye la cadena final únicamente cuando invocamos `.toString()`.

3.2 Operaciones Matemáticas Avanzadas con `dart:math`

La librería principal `dart:math` ofrece constantes matemáticas universales, funciones trigonométricas, algoritmos de conversión y un generador de números pseudoaleatorios.

Métodos y Funciones Frecuentes

- **Constantes:** `pi`, `e`, `sqrt2`.
 - **Redondeo y Clamping:**
 - `.floor()`: Redondea hacia abajo al entero más cercano.
 - `.ceil()`: Redondea hacia arriba al entero más cercano.
 - `.round()`: Redondea al entero más próximo según el valor decimal.
 - `.clamp(num lowerLimit, num upperLimit)`: Restringe un número dentro de un rango definido.
 - **Generación Aleatoria (Random):** Permite generar enteros dentro de un rango estricto mediante `.nextInt(int max)` o valores de punto flotante en el rango de `[0.0, 1.0)` con `.nextDouble()`.
-

3.3 Manejo de Entrada de Datos con `dart:io`

Para interactuar con la línea de comandos en entornos de ejecución nativos (como la máquina virtual de Dart), importamos la librería `dart:io`.

Lectura desde Teclado con `stdin.readLineSync()`

El método `stdin.readLineSync()` detiene la ejecución del programa hasta que el usuario ingresa una secuencia de caracteres y presiona `Enter`.

```
import 'dart:io';

void main() {
  stdout.write('Ingrese su nombre: ');
  // readLineSync() retorna un String? porque la entrada puede ser nula (EOF)
  String? entrada = stdin.readLineSync();
}
```

Atención de Seguridad Nula: Debido a que la lectura desde la consola puede retornar un valor nulo (por ejemplo, si se interrumpe la entrada con `Ctrl + C` o un fin de archivo `EOF`), siempre debemos manejar el tipo nullable `String?` y validar la cadena antes de procesarla o hacer un parsing a un tipo numérico (`int.parse()` / `double.tryParse()`).

3.4 Ejemplo

Creen el archivo `bin/d3_string_math_io.dart` en VS Code. El siguiente programa demuestra la interactividad mediante entrada por consola, la limpieza de cadenas mediante Expresiones Regulares, y la aplicación de funciones matemáticas y aleatorias.

```
/// Módulo D3: Objetos String, Math y Entrada de Datos (dart:io)
/// Demostración práctica de procesamiento interactivo y sanitización de datos.

import 'dart:io';
import 'dart:math';

void main() {
  print('=====');
  print('      SISTEMA INTERACTIVO DE VALIDACIÓN Y CÁLCULO - ESCOM      ');
  print('=====');

  // 1. Lectura interactiva de la boleta
  stdout.write('\nIngrese su número de boleta: ');
  final String? boletaRaw = stdin.readLineSync();

  // Sanitización de la cadena: remover espacios y caracteres no numéricos
  final String boletaLimpia = (boletaRaw ?? '').trim()
```

```

        .replaceAll(RegExp(r'^0-9'), '');

if (boletaLimpia.isEmpty || boletaLimpia.length != 10) {
    print('Error: La boleta ingresada es inválida. Debe contener 10 dígitos.');
```

return;

```

}

print('Boleta procesada correctamente: $boletaLimpia');
```

// 2. Lectura y parsing de datos numéricos

```

stdout.write('Ingrese la calificación del examen parcial (0 - 10): ');
final String? calificacionInput = stdin.readLineSync();

// Uso de tryParse para evitar excepciones de runtime si el input no es numérico
final double? calificacionParsed = double.tryParse(calificacionInput ?? '');

if (calificacionParsed == null) {
    print('Error: Debe ingresar un valor numérico válido.');
```

return;

```

}

// Restringimos el rango numérico usando clamp() entre 0.0 y 10.0
final double calificacionFinal = calificacionParsed.clamp(0.0, 10.0);
print('Calificación ajustada (clamp 0-10): $calificacionFinal');
```

// 3. Uso de la librería dart:math

```

print('\n--- Procesamiento Matemático (dart:math) ---');
```

// Redondeo de la calificación

```

print('Valor truncado (floor): ${calificacionFinal.floor()}');
print('Valor redondeado hacia arriba (ceil): ${calificacionFinal.ceil()}');
print('Valor redondeado estándar (round): ${calificacionFinal.round()}');
```

// Generación de un Token Criptográfico/Aleatorio de Seguridad

```

final Random random = Random();
final int idTransaccion = random.nextInt(900000) + 100000; // Número de 6
dígitos
final double factorPonderacion = random.nextDouble(); // Valor entre 0.0 y
1.0

// 4. Formateo y construcción final de la salida con StringBuffer
final StringBuffer reporteBuffer = StringBuffer()
    ..write('\n=====')
    ..write('RESUMEN DE REGISTRO')
    ..write('=====')
    ..write('Boleta Estudiante: $boletaLimpia\n')
    ..write('Año de Ingreso: 20${boletaLimpia.substring(0, 2)}\n')
    ..write('Nota Registrada: $calificacionFinal\n')
    ..write('Factor Aleatorio: ${(factorPonderacion *
100).toStringAsFixed(2)}%\n')
    ..write('ID de Transacción: ESCOM-$idTransaccion\n')
    ..write('Constante Math Pi: ${pi.toStringAsFixed(4)}\n')
    ..write('=====');
```

```
print(reporteBuffer.toString());  
}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d3_string_math_io.dart
```

3.5 Ejercicios Prácticos

Resuelvan los siguientes dos problemas en su entorno local para consolidar la manipulación de cadenas, el uso de funciones de `dart:math` y la lectura interactiva.

Ejercicio 1: Generador de Contraseñas y Token de Seguridad

Crea un script interactivo en Dart que genere un token seguro para una sesión de usuario:

1. Pide al usuario su nombre completo mediante `stdin.readLineSync()`.
2. Elimina los espacios innecesarios y convierte la cadena a mayúsculas.
3. Extrae las primeras tres letras del nombre utilizando `.substring()`.
4. Utiliza la clase `Random` de `dart:math` para generar un código numérico aleatorio de 4 dígitos.
5. Imprime en consola un token formateado combinando la subcadena del nombre, el año actual y el código aleatorio (Ejemplo: `OZZ-2026-8492`).

Ejercicio 2: Calculadora de Distancia Geodésica Simplificada

Crea un programa que calcule la distancia en línea recta entre dos puntos cartesianos representando coordenadas en un mapa:

1. Solicita por consola las coordenadas (x_1, y_1) del Punto A y (x_2, y_2) del Punto B (valores decimales `double`).
2. Implementa la fórmula de la distancia euclidiana:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. Para la potencia utiliza la función `pow()` y para la raíz cuadrada utiliza `sqrt()` de la librería `dart:math`.
 4. Muestra la distancia calculada redondeada a 2 decimales usando `.toStringAsFixed(2)`.
-